Setup Active-Active Galera Cluster Simulation

Introduction to Database Clustering

Database clustering is a technology that combines multiple database servers into a single logical system to improve availability, reliability, and fault tolerance. In a clustered environment, multiple database servers work together and share the same data.

Database clustering ensures that if one database server fails, the remaining servers continue to provide services without interrupting applications.

Galera Cluster is a synchronous multi-master replication technology used with MariaDB and MySQL databases. It enables real-time replication between database servers, ensuring that all nodes maintain identical data.

Introduction to Galera Cluster

Galera Cluster is an open-source synchronous replication solution for MariaDB and MySQL databases. It allows multiple database nodes to operate simultaneously while maintaining data consistency.

Unlike traditional master-slave replication, Galera Cluster provides multi-master functionality, allowing read and write operations on any active node.

Galera Cluster offers several advantages including:

- High Availability
- Fault Tolerance
- Data Consistency
- Automatic Replication
- Load Distribution
- No Single Point of Failure

Importance, Benefits and Applications of Galera Cluster

Traditional database systems typically depend on a single database server. If that server fails, applications may experience downtime and service interruptions. Galera Cluster addresses this challenge by maintaining multiple active database servers that work together as a synchronized cluster, ensuring continuous availability and data consistency.

Benefits: High availability, real-time synchronization, automatic failover, protection against data loss, improved reliability, read scalability, and reduced latency.

Applications: Banking systems, e-commerce platforms, cloud databases, enterprise applications, high-availability environments, and other business-critical systems where continuous database access is essential.

What is Active-Active Cluster?

An Active-Active Cluster is a cluster configuration in which all database nodes remain operational simultaneously.

Every node can process database requests and automatically replicate data to other nodes.

If one server fails, the remaining servers continue serving requests.

What is MariaDB?

MariaDB is an open-source relational database management system (RDBMS) developed as a fork of MySQL. It is used to store, manage, and retrieve data efficiently. MariaDB provides high performance, reliability, security, and scalability, making it suitable for web applications, enterprise systems, and cloud environments.

MariaDB supports SQL and advanced features such as replication, clustering, backup, and high availability. It is widely used in web applications, e-commerce platforms, banking systems, and enterprise databases. In this project, MariaDB is used because it supports Galera Cluster technology, which enables real-time data synchronization and high availability in an Active-Active database environment.

What is HAProxy?

HAProxy (High Availability Proxy) is an open-source load balancer and proxy server used to distribute network traffic among multiple servers. It improves the availability, reliability, and performance of applications by forwarding client requests to healthy backend servers.

In this project, HAProxy acts as a load balancer between the client and the Galera database nodes. It distributes database requests among the available database servers and automatically redirects traffic if one node fails. This ensures continuous database availability and fault tolerance in the Active-Active Galera Cluster environment.

Since HAProxy supports high availability and load balancing, it is widely used in web servers, database clusters, cloud environments, and enterprise applications.

Architecture of Galera Cluster

Galera Cluster follows a synchronous multi-master architecture.

The cluster consists of:

- Database Nodes
- Replication Layer
- Load Balancer
- Client Applications

Components of Galera Cluster

Component	Description	Function
MariaDB	Database Server	Stores data
Galera	Replication Engine	Synchronizes data
HAProxy	Load Balancer	Distributes traffic
Database Nodes	Cluster Servers	Maintain database copies
Rsync	State Transfer	Synchronizes nodes
Client Application	User Access	Connects to database

Table: Components of Galera Cluster.

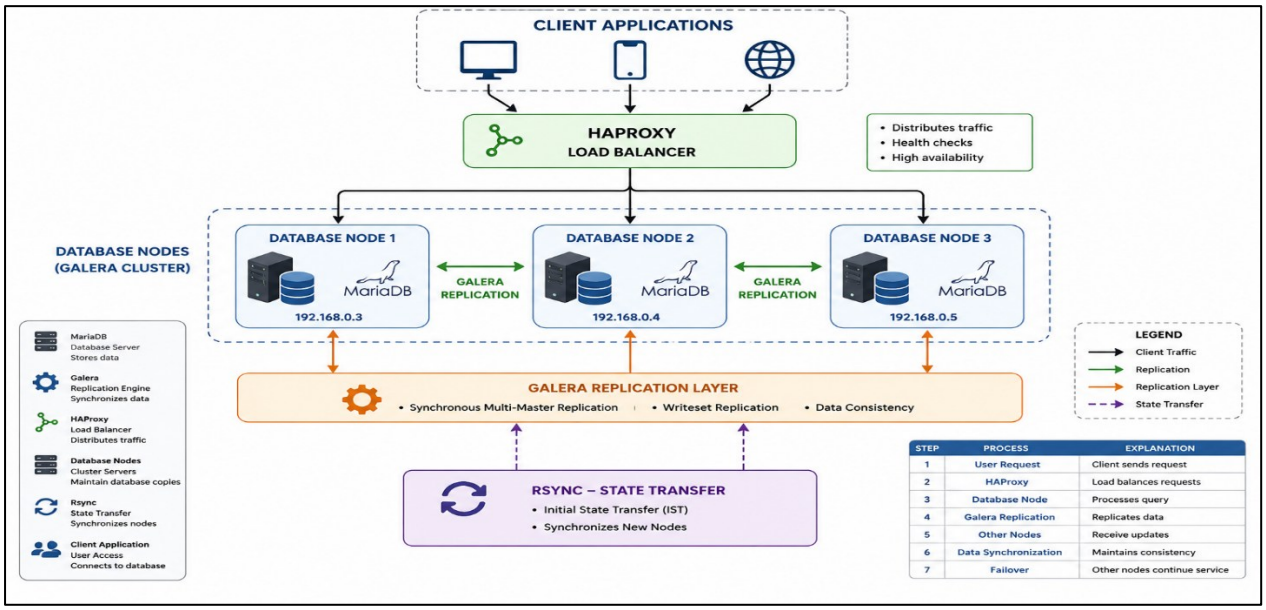


Figure: Galera Cluster Architecture

5. Working of Galera Cluster

Step	Process	Explanation
1	User Request	Client sends request
2	HAProxy	Load balances requests
3	Database Node	Processes query
4	Galera Replication	Replicates data
5	Other Nodes	Receive updates
6	Data Synchronization	Maintains consistency
7	Failover	Other nodes continue service

Table: Working of Galera Cluster

Project Overview

This project demonstrates the implementation of an Active-Active Galera Cluster using MariaDB and HAProxy on VMware Cloud Director.

The cluster consists of:

- Database Node 1
- Database Node 2
- Database Node 3
- HAProxy Load Balancer

The project provides high availability, automatic replication, and database failover capabilities.

Project Objectives

- Deploy Ubuntu virtual machines.
- Install MariaDB and Galera Cluster.
- Configure synchronous replication.
- Implement Active-Active clustering.
- Configure HAProxy load balancing.
- Test database synchronization.
- Verify failover capabilities.
- Demonstrate high availability.

System Specifications

Component	**Details**
Platform	VMware Cloud Director
Operating System	Ubuntu 22.04 LTS
Database	MariaDB 10.6
Cluster Technology	Galera Cluster
Load Balancer	HAProxy
Replication	Synchronous
SSH Port	2232

Table: System Specifications

Project Architecture

VM Name	**IP Address**	**Role**
HAProxyLoadBalancer	192.168.0.2	Load Balancer
GaleraDBNode1	192.168.0.3	Database Node

GaleraDBNode2	192.168.0.4	Database Node
GaleraDBNode3	192.168.0.5	Database Node

Table: Project Architecture Configuration

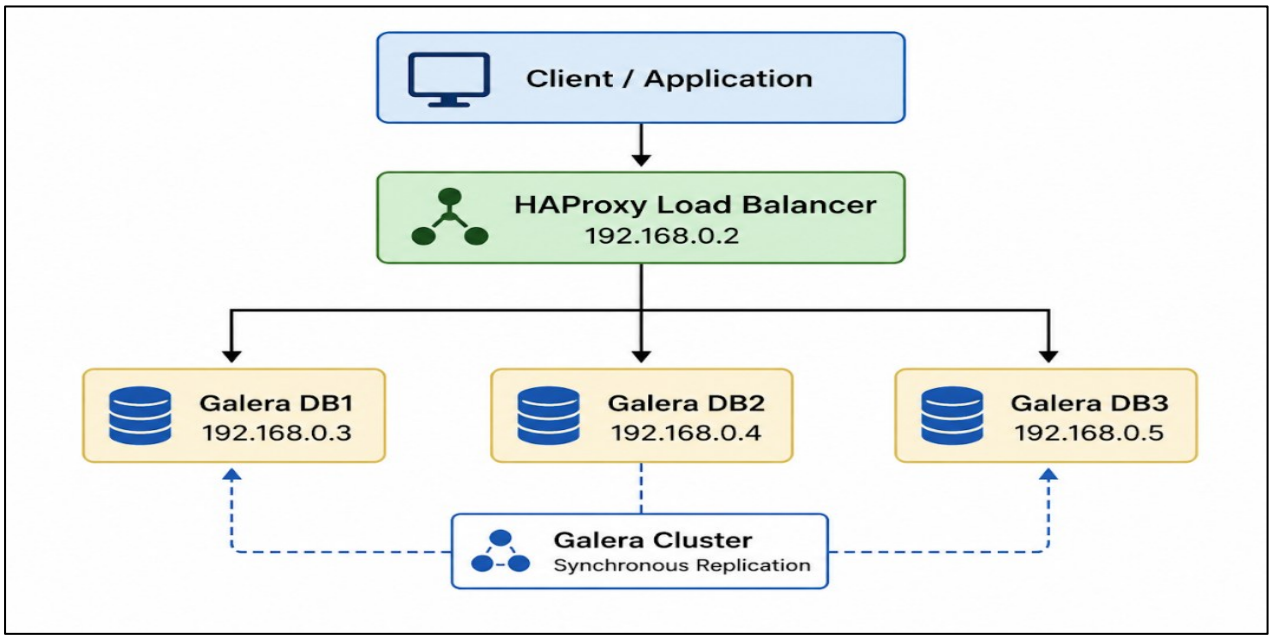


Figure: Project Architecture

IMPLEMENTATION

1. Create Virtual Machines

VM Name	CPU	RAM	Disk
HAProxyLoadBalancer	4	8 GB	40 GB
GaleraDBNode1	4	8 GB	40 GB
GaleraDBNode2	4	8 GB	40 GB
GaleraDBNode3	4	8 GB	40 GB

Table: VM Configuration

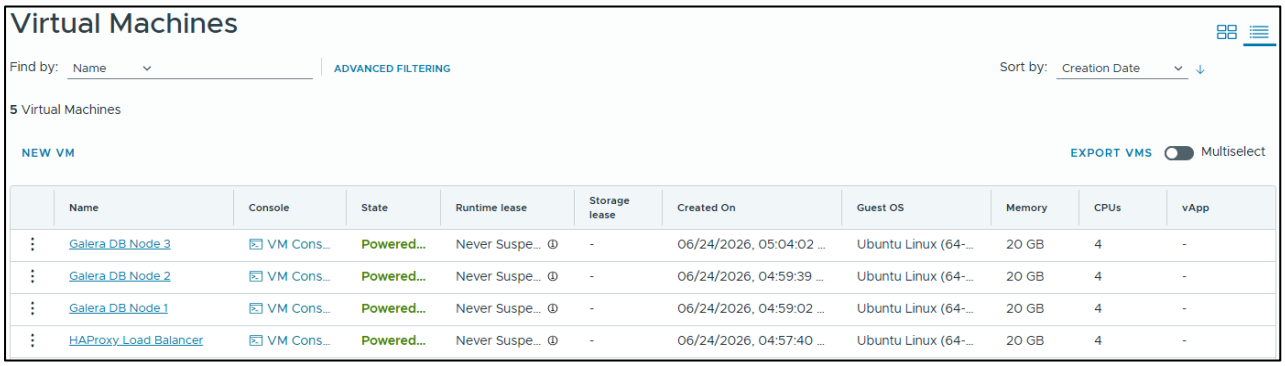


Figure: VM Creation

3. Install MariaDB and Galera

Step	Command
Update Packages	apt update
Install MariaDB	apt install mariadb-server
Install Galera	apt install galera-4
Install Rsync	apt install rsync

Table: Installation Commands

```
root@GaleraDBNode3:~# systemctl status mariadb
● mariadb.service - MariaDB 10.6.23 database server
     Loaded: loaded (/lib/systemd/system/mariadb.service; disabled; vendor pres>
     Active: active (running) since Thu 2026-06-25 07:20:12 UTC; 2h 42min ago
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
    Process: 23354 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /va>
    Process: 23357 ExecStartPre=/bin/sh -c systemctl unset-environment _WSREP_S>
    Process: 23359 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] &&>
    Process: 23614 ExecStartPost=/bin/sh -c systemctl unset-environment _WSREP_>
    Process: 23616 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0>
   Main PID: 23436 (mariadbd)
     Status: "Taking your SQL requests now..."
      Tasks: 13 (limit: 157349)
     Memory: 88.1M
        CPU: 7.406s
     CGroup: /system.slice/mariadb.service
             └─23436 /usr/sbin/mariadbd --wsrep_start_position=12ced38d-7060-11>
```

Figure: MariaDB Installed.

4. Stop MariaDB Service

Run on all database nodes.

systemctl stop mariadb

Purpose: Prevent MariaDB from starting before configuration.

5. Configure Galera Cluster

Configuration file:

/etc/mysql/mariadb.conf.d/60-galera.cnf

Important parameters:

- wsrep_on
- wsrep_cluster_name
- wsrep_cluster_address
- wsrep_node_address
- wsrep_node_name

```
root@GaleraDBNode3:~# cat /etc/mysql/mariadb.conf.d/60-galera.cnf
[mysqld]
bind-address=0.0.0.0

binlog_format=ROW
default_storage_engine=InnoDB
innodb_autoinc_lock_mode=2

wsrep_on=ON
wsrep_provider=/usr/lib/galera/libgalera_smm.so
wsrep_cluster_name="galera_cluster"
wsrep_cluster_address="gcomm://192.168.0.3,192.168.0.4,192.168.0.5"

wsrep_node_address="192.168.0.5"
wsrep_node_name="db3"

wsrep_sst_method=rsync
root@GaleraDBNode3:~#
```

Figure: Galera Configuration

6. Configure Cluster Nodes and Bootstrap Galera Cluster

VM / Node	Configuration / Command	Purpose
GaleraDBNode2 (192.168.0.4)	wsrep_node_address=192.168.0.4 wsrep_node_name=db2	Identifies and configures the second database node.
GaleraDBNode3 (192.168.0.5)	wsrep_node_address=192.168.0.5 wsrep_node_name=db3	Identifies and configures the third database node.
GaleraDBNode1 (DB1)	galera_new_cluster	Bootstraps and creates the initial Galera Cluster.
Verification	systemctl status mariadb	Verifies that the MariaDB service and cluster are running successfully.

Table: Configure Cluster Nodes

7. Verify Cluster Size

Run on DB1.

mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"

Expected output:

wsrep_cluster_size = 1

```
root@GaleraDBNode1:~# mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
+--------------------+-------+
| Variable_name      | Value |
+--------------------+-------+
| wsrep_cluster_size | 1     |
+--------------------+-------+
root@GaleraDBNode1:~#
```

Figure: Verify Cluster Size

8. Join DB2 to Cluster

Run on DB2.

systemctl start mariadb

Verify:

mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"

Expected:

wsrep_cluster_size = 2

```
root@GaleraDBNode2:~# systemctl start mariadb
root@GaleraDBNode2:~#  mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
+--------------------+-------+
| Variable_name      | Value |
+--------------------+-------+
| wsrep_cluster_size | 2     |
+--------------------+-------+
root@GaleraDBNode2:~#
```

Figure: Join DB2 to Cluster

9. Join DB3 to Cluster

Run on DB3.

systemctl start mariadb

Verify:

mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"

Expected:

wsrep_cluster_size = 3

```
root@GaleraDBNode3:~#  systemctl start mariadb
root@GaleraDBNode3:~# mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
+--------------------+-------+
| Variable_name      | Value |
+--------------------+-------+
| wsrep_cluster_size | 3     |
+--------------------+-------+
root@GaleraDBNode3:~#
```

Figure: Join DB3 to Cluster

10. Verify Cluster Status

Run on any database node.

mysql -e "SHOW STATUS LIKE 'wsrep_cluster_status';"

Expected: Primary

Check readiness:

mysql -e "SHOW STATUS LIKE 'wsrep_ready';"

Expected: ON

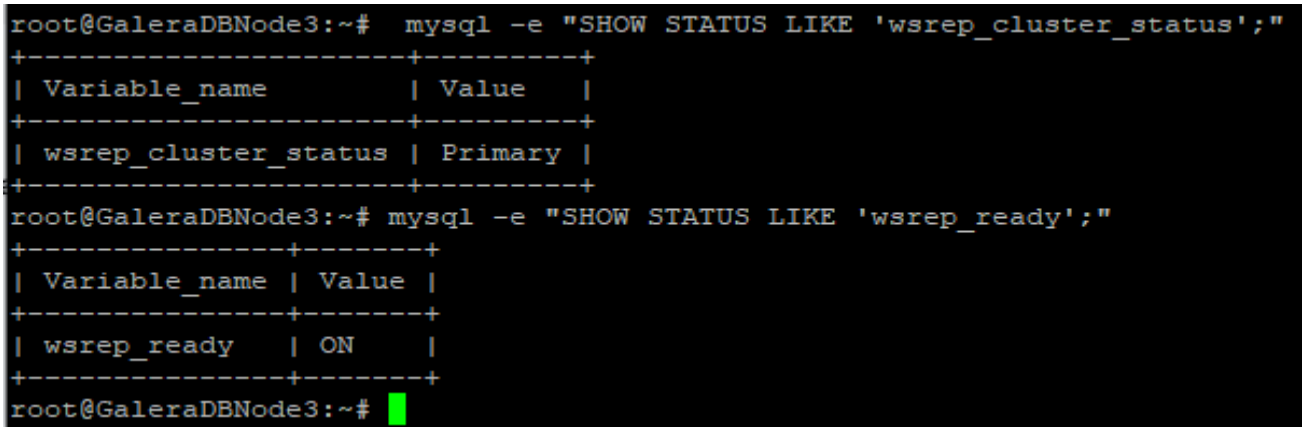


Figure: Verify Cluster Status

11. Install HAProxy

Run on HAProxyLoadBalancer.

apt update

apt install -y haproxy mariadb-client

Purpose:

- Install load balancer.
- Install MariaDB client.

12. Configure HAProxy

Edit:

nano /etc/haproxy/haproxy.cfg

Add:

frontend galera_frontend

bind *:3306

default_backend galera_backend

backend galera_backend

balance roundrobin

option tcp-check

server db1 192.168.0.3:3306 check

server db2 192.168.0.4:3306 check

server db3 192.168.0.5:3306 check

Purpose: Load balances database traffic.

```
root@HAProxyLoadBalancer:~#  cat /etc/haproxy/haproxy.cfg
global
    log /dev/log local0
    maxconn 4096
    daemon

defaults
    log global
    mode tcp
    option tcplog
    timeout connect 10s
    timeout client 1m
    timeout server 1m

frontend galera_frontend
    bind *:3306
    default_backend galera_backend

backend galera_backend
    balance roundrobin
    option tcp-check
    server db1 192.168.0.3:3306 check
    server db2 192.168.0.4:3306 check
    server db3 192.168.0.5:3306 check
root@HAProxyLoadBalancer:~#
```

Figure: Configure HAProxy

13. Start HAProxy

systemctl restart haproxy

systemctl enable haproxy

systemctl status haproxy

Purpose: Start and verify HAProxy service.

```
root@HAProxyLoadBalancer: ~
Connection to 192.168.0.5 3306 port [tcp/mysql] succeeded!
root@HAProxyLoadBalancer:~# systemctl restart haproxy
systemctl status haproxy --no-pager
● haproxy.service - HAProxy Load Balancer
     Loaded: loaded (/lib/systemd/system/haproxy.service; enabled; vendor preset
: enabled)
     Active: active (running) since Thu 2026-06-25 06:48:01 UTC; 6ms ago
       Docs: man:haproxy(1)
             file:/usr/share/doc/haproxy/configuration.txt.gz
    Process: 3433 ExecStartPre=/usr/sbin/haproxy -Ws -f $CONFIG -c -q $EXTRAOPTS
 (code=exited, status=0/SUCCESS)
   Main PID: 3435 (haproxy)
      Tasks: 5 (limit: 23840)
     Memory: 5.0M
        CPU: 24ms
     CGroup: /system.slice/haproxy.service
             ├─3435 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/h…
             └─3438 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/h…
```

Figure: Start HAProxy

14. Create Test Database

Run on HaProxy.

mysql

CREATE DATABASE projectdb;

USE projectdb;

CREATE TABLE students(

id INT PRIMARY KEY AUTO_INCREMENT,

name VARCHAR(50)

);

INSERT INTO students(name)

VALUES('Chand');

SELECT * FROM students;

Purpose: Verify replication.

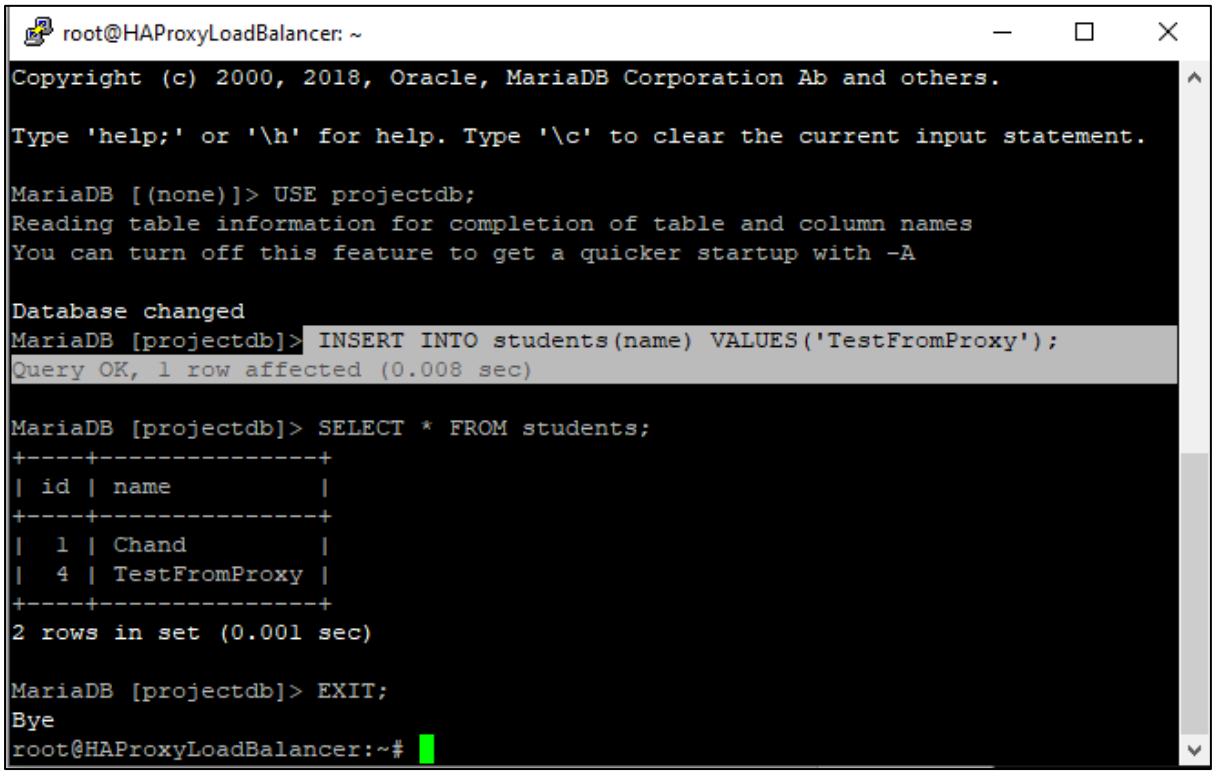


Figure: Create Test Database

15. Verify Replication

Run on DB1:

mysql -e "SELECT * FROM projectdb.students;"

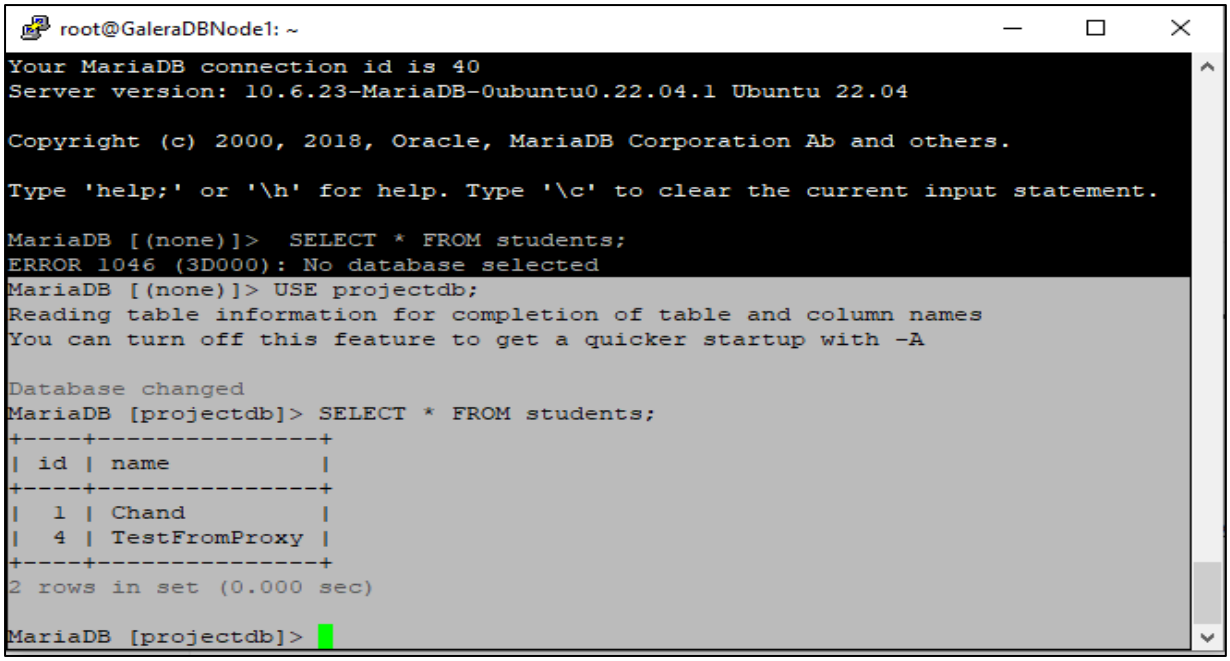


Figure: Verify Replication

16. Failover Testing

Stop DB3.

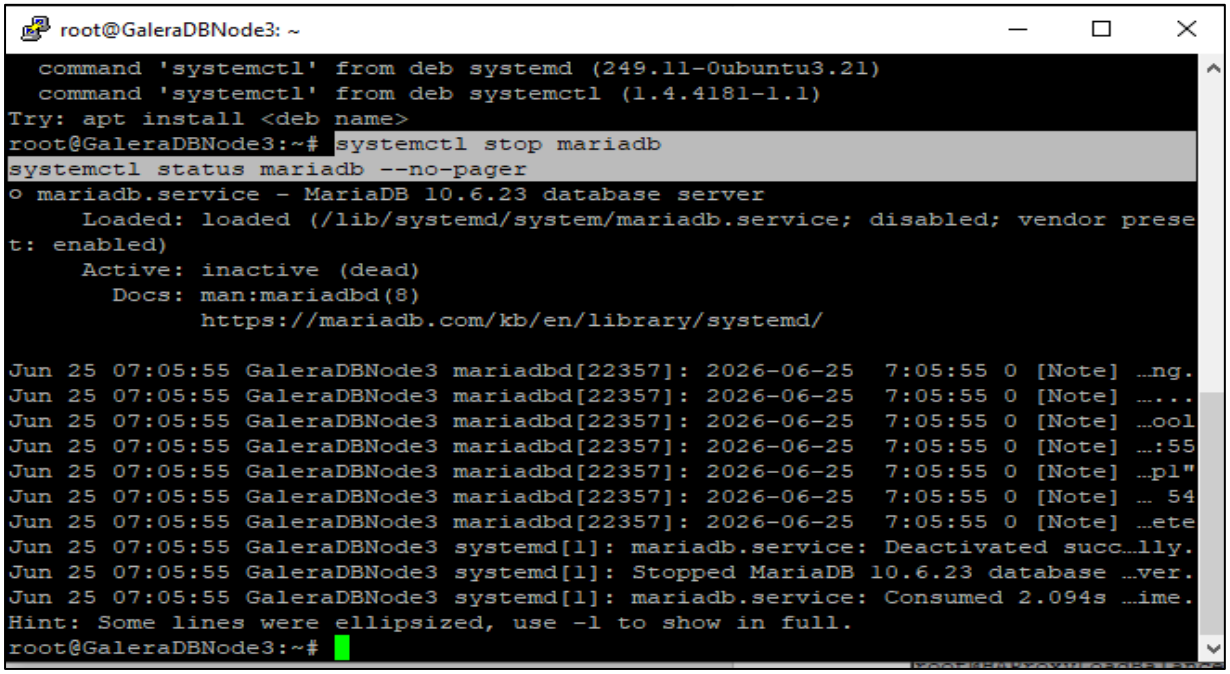
systemctl stop mariadb

Check cluster size on DB1.

mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"

Expected: wsrep_cluster_size = 2

Purpose: Verify high availability.



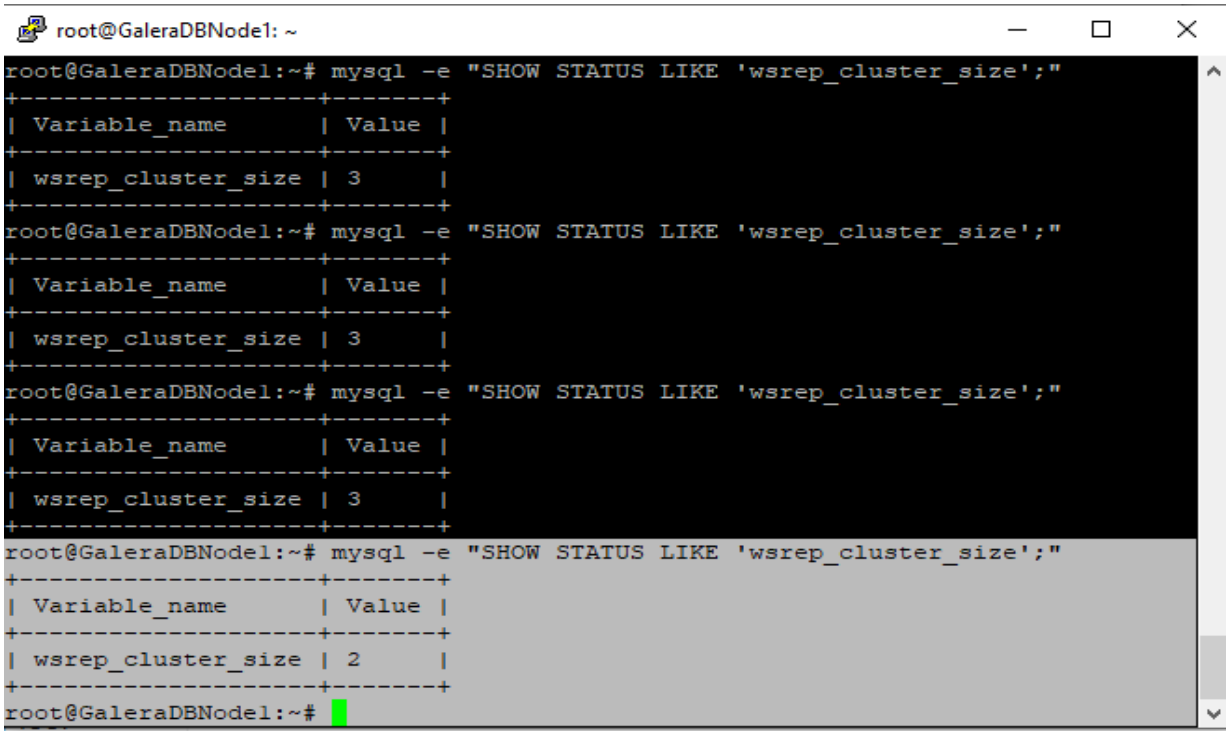


Figure: Failover Testing

17. Recover Failed Node

Run on DB3.

systemctl start mariadb

Verify:

mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"

Expected: wsrep_cluster_size = 3

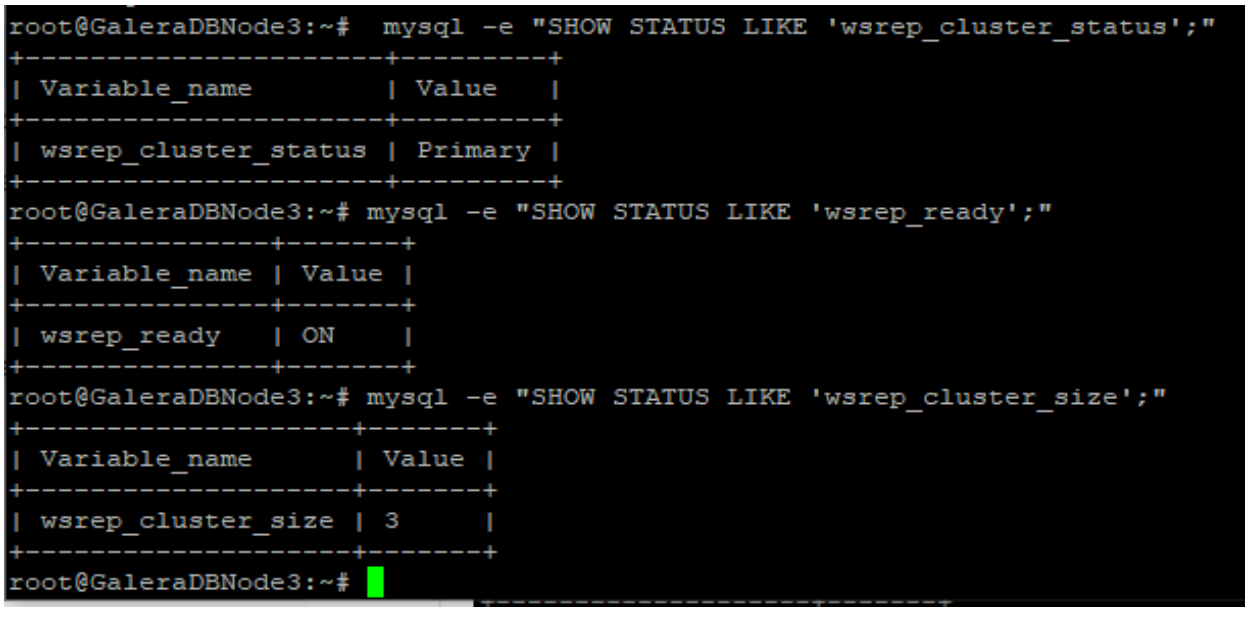


Figure: Recover Failed Node

Challenges Encountered

Issue	Resolution
Apt lock error	Stopped unattended-upgrades
MariaDB installation failed	Reinstalled packages
wsrep_cluster_size = 0	Corrected Galera configuration
HAProxy backend down	Opened firewall ports
Node synchronization issue	Restarted MariaDB

Table: Challenges Encountered

Project Repository

GitHub Repository:

https://github.com/2121-shivi/Galera-Cluster

Conclusion

The Active-Active Galera Cluster project was successfully implemented using VMware Cloud Director and Ubuntu Linux. MariaDB Galera Cluster provided synchronous database replication between three database nodes, while HAProxy distributed database traffic among available nodes.

The project provided practical knowledge of database clustering, load balancing, high availability, failover mechanisms, and enterprise database infrastructure.

The implementation demonstrated how modern organizations can achieve reliable and fault-tolerant database environments using Galera Cluster technology.